React useEffect Hook: Complete Guide

What is useEffect?

useEffect is a React Hook that lets you perform side effects in function components. It serves as a replacement for lifecycle methods (componentDidMount, componentDidUpdate, and componentWillUnmount) from class components.

Why do we need useEffect?

1. Handle Side Effects

React components should be pure functions of props and state. However, real applications need to perform side effects like:

- Data fetching
- Setting up subscriptions
- Manually changing the DOM
- Timers and intervals

2. Separate Concerns

useEffect helps separate side effects from the rendering logic, making components more predictable and easier to test.

3. Prevent Memory Leaks

It provides a built-in cleanup mechanism to avoid memory leaks and unwanted behavior.

How to use useEffect?

Basic Syntax

jsx

```
useEffect(() => {
  // Side effect code here
  return () => {
    // Cleanup code (optional)
  };
}, [dependencies]); // Dependency array
```

useEffect Dependency Patterns :

1. No Dependency Array (Run after every render)

jsx

```
useEffect(() => {
  console.log('This runs after every render');
});
```

2. Empty Dependency Array (Run once after mount)

jsx

```
useEffect(() => {
  console.log('This runs only once after initial render');

  return () => {
    console.log('Cleanup on unmount');
  };
}, []);
```

3. With Dependencies (Run when dependencies change)

jsx

```
useEffect(() => {
  console.log('This runs when count changes:', count);
}, [count]);
```

Common Use Cases

Data Fetching

jsx

```
useEffect(() => {
  const fetchData = async () => {
    try {
      const response = await fetch(`/api/users/${userId}`);
      const data = await response.json();
      setUser(data);
    } catch (error) {
      setError(error.message);
    }
  };
    fetchData();
}, [userId]);
```

Event Listeners

jsx

```
useEffect(() => {
  const handleResize = () => {
    setWindowSize({
      width: window.innerWidth,
      height: window.innerHeight
    });
  };
  window.addEventListener('resize', handleResize);
 return () => {
    window.removeEventListener('resize', handleResize);
  };
```

```
}, []);
```

Timers

jsx

```
useEffect(() => {
  const timer = setInterval(() => {
    setSeconds(prev => prev + 1);
  }, 1000);

  return () => clearInterval(timer);
}, []);
```

Interview Questions & Answers :

Q1: What is the purpose of the dependency array in useEffect?

Answer:
The dependency array controls when the effect runs:

- **No array**: Effect runs after every render
- **Empty array []**: Effect runs only once after initial mount
- **With values [a, b]**: Effect runs when any value in the array changes

It helps optimize performance by preventing unnecessary effect executions.

Q2: How do you prevent infinite loops in useEffect?

Answer:
Infinite loops occur when:

1. Effect updates state that triggers re-render
2. Effect has no dependencies or wrong dependencies

jsx

```
//  Causes infinite loop
useEffect(() => {
  setCount(count + 1);
});


//  empty dependency array
useEffect(() => {
  // One-time setup
}, []);


// proper dependencies
useEffect(() => {
  setCount(prevCount => prevCount + 1);
}, [specificDependency]);
```

Q3: What is the cleanup function and when is it called?

Answer:
The cleanup function:

- Runs before the component unmounts
- Runs before re-running the effect (if dependencies change)
- Used to clean up subscriptions, timers, event listeners

jsx

```
useEffect(() => {
  const subscription = api.subscribe();


  return () => {
    subscription.unsubscribe(); // Cleanup
  };
}, []);
```

Q4: What's the difference between useEffect and useLayoutEffect?

Answer:

- **useEffect**: Runs asynchronously after browser paint (non-blocking)
- **useLayoutEffect**: Runs synchronously after DOM mutations but before paint (blocking)

When to use useLayoutEffect:

- When you need to read layout and synchronously re-render
- When visual updates would cause flickering

Q5: How do you handle async operations in useEffect?

Answer:
Option 1: Async function inside effect

jsx

```
useEffect(() => {
  const fetchData = async () => {
    const data = await api.call();
```

```
    setState(data);
  };

  fetchData();
}, []);
```

Option 2: Using .then()

```
jsx
useEffect(() => {
  api.call().then(data => setState(data));
}, []);
```

Important: useEffect cannot be an async function directly because it expects a cleanup function or nothing in return.

Q6: How do you conditionally run effects?

Answer:
Option 1: Condition inside effect

```
jsx
useEffect(() => {
  if (shouldFetch) {
    fetchData();
  }
}, [shouldFetch, fetchData]);
```

Option 2: Conditional dependencies

```
jsx
useEffect(() => {
  fetchData();
}, [userId]); // Only runs when userId changes
```

Q7: What are common mistakes with useEffect?

Answer:

1. **Missing dependencies**
2. **Infinite re-renders**
3. **Stale closures**
4. **No cleanup for subscriptions**
5. **Using objects/arrays in dependencies without memoization**

Q8: How do you fix stale closures in useEffect?

Answer:
Problem:

jsx

```
useEffect(() => {
  const timer = setInterval(() => {
    console.log(count); // Always logs initial count
  }, 1000);

  return () => clearInterval(timer);
}, []);
```

Solution 1: Include dependency

jsx

```
useEffect(() => {
  const timer = setInterval(() => {
    console.log(count);
  }, 1000);

  return () => clearInterval(timer);
}, [count]); // Re-create when count changes
```

Solution 2: Use functional updates

```jsx
useEffect(() => {
  const timer = setInterval(() => {
    setCount(prev => prev + 1); // Always gets latest state
  }, 1000);

  return () => clearInterval(timer);
}, []);
```

Q9: When should you use multiple useEffect hooks?

Answer:
Use multiple useEffect hooks to separate unrelated logic:

```jsx
// Data fetching
useEffect(() => {
  fetchUser(userId);
}, [userId]);

// Event listeners
useEffect(() => {
  window.addEventListener('resize', handleResize);
  return () => window.removeEventListener('resize', handleResize);
}, []);

// Document title
useEffect(() => {
  document.title = `User ${userId}`;
}, [userId]);
```

Q10: How do you optimize expensive operations in useEffect?

Answer:

Use useMemo/useCallback with useEffect:

jsx

```
const expensiveValue = useMemo(() => {
  return computeExpensiveValue(a, b);
}, [a, b]);

const fetchData = useCallback(() => {
  // fetch logic
}, [dependency]);

useEffect(() => {
  fetchData();
}, [fetchData]);
```

Advanced Patterns

Conditional Effects

jsx

```
useEffect(() => {
  if (!userId) return;

  // Effect logic here
}, [userId]);
```

Sequential Effects

jsx

```
useEffect(() => {
  // First effect
}, [dep1]);
```

```
useEffect(() => {
  // Second effect that depends on first
}, [dep2]);
```

SYNTAX PRATICE :

1. useEffect(()=>{ },[depandancies])
2. useEffect(()=>{ },[depandancies])
3. useEffect(()=>{ },[depandancies])
4. useEffect(()=>{ } ,[depandancies])
5. useEffect(()=>{ },[depandancies])
6. useEffect(()=>{ },[depandancies])
7. useEffect(()=>{ },[depandancies])
8. useEffect(()=>{ },[depandancies])
9. useEffect(()=>{ },[depandancies])
10. useEffect(()=>{ },[depandancies])
11. useEffect(()=>{ },[depandancies])
12. useEffect(()=>{ },[depandancies])
13. useEffect(()=>{ },[depandancies])
14. useEffect(()=>{ },[depandancies])
15. useEffect(()=>{ },[depandancies])
16. useEffect(()=>{ },[depandancies])
17. useEffect(()=>{ },[depandancies])
18. useEffect(()=>{ },[depandancies])
19. useEffect(()=>{ },[depandancies])
20. useEffect(()=>{ },[depandancies])
21. useEffect(()=>{ },[depandancies])
22. useEffect(()=>{ },[depandancies])
23. useEffect(()=>{ },[depandancies])
24. useEffect(()=>{ },[depandancies])